@linux_dev_ops

NGINX SERIES

PART 5

Where NGINX Fits in Production

NGINX is the smart gatekeeper of your infrastructure.

Fast • Secure
Reliable • Scalable

WHY PRODUCTION TEAMS CHOOSE NGINX

- Handles thousands of concurrent connections easily.
- Low memory & CPU footprint.
- Flexible & powerful configuration.
- Excellent reverse proxy & load balancing.
- Strong logging & monitoring integrations.
- Huge community & battle-tested in production.

COMMON BEGINNER MISTAKES

- Editing configuration without testing.
- Forgetting upstream timeouts.
- Exposing backend services directly.
- Ignoring access & error logs.
- Restarting instead of reloading safely.

IMPORTANT COMMANDS

```
# Test configuration
sudo nginx -t

# Reload configuration (zero downtime)
sudo systemctl reload nginx

# Check status
sudo systemctl status nginx
```

COMING NEXT

Part 2: Reverse Proxy & Load Balancing

- Upstreams & Load Balancing Methods
- Health Checks
- Timeouts Explained
- Real Production Examples

NGINX is not just a web server – it's the backbone that keeps modern systems fast, secure & reliable.

Stay tuned & Keep Learning!

@linux_dev_ops

NGINX SERIES

PART 3

How Does NGINX Work Internally?

Let's see what happens when a request hits your Nginx server.

REQUEST FLOW

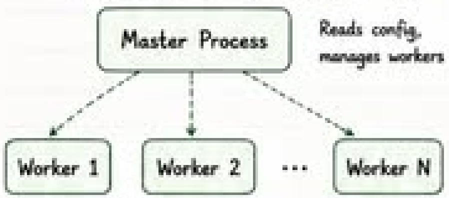
WHAT HAPPENS BEHIND THE SCENES?

1. Master Process starts and reads the configuration.
2. Master Process creates Worker Processes.
3. Worker Processes handle incoming connections using an event loop.
4. NGINX decides how to handle the request based on the configuration.
5. It can serve a static file, proxy to backend, use cache, or balance the load.
6. Response is returned back to the client.

Key Point

NGINX handles many connections with a small number of processes using an event-driven, asynchronous model.

NGINX PROCESS ARCHITECTURE



Each worker uses an event loop to handle many connections concurrently.

SIMPLE CONFIG EXAMPLE

```
server {
    listen 80;                      # Listen on port 80
    server_name example.com;        # Domain

    location /api/ {
        proxy_pass http://backend;  # Forward to backend
    }
}
```

NOTE

Pro Tip

@linux_dev_ops

NGINX SERIES

PART 2

Why Was NGINX Created?

Traditional web servers struggled when thousands of clients connected simultaneously.

Built for high concurrency & low overhead

TRADITIONAL MODEL

Process/Thread per Connection

Connection 1 → Process/Thread

Connection 2 → Process/Thread

Connection 3 → Process/Thread

⋮

More connections → More Processes

↓

More memory usage

Slower server ☹

VS

NGINX MODEL (EVENT-DRIVEN)

Event Loop + Non-blocking I/O

Event Loop

Conn 1 · Conn 2 · Conn 3 · … · Conn N

↓

Many connections

↓

Low memory usage

High performance ☺

Key Takeaway

NGINX was designed for high concurrency, predictable memory usage, and efficient network I/O.

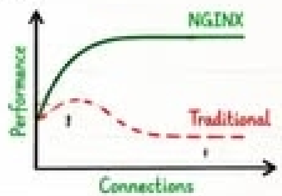
THE C10K PROBLEM

C10K = 10,000 concurrent connections

Traditional servers couldn't handle 10K+ concurrent connections efficiently. NGINX solved this with an asynchronous, non-blocking architecture.

BENEFITS OF NGINX ARCHITECTURE

- ☑ Handles thousands of connections with ease
- ☑ Uses fewer resources (CPU & Memory)
- ☑ Non-blocking & event-driven
- ☑ Highly scalable
- ☑ Stable under heavy load



NGINX isn't just a web server – it's a high performance engine for modern applications.

Swipe
to see how NGINX works internally →

@linux_dev_ops

NGINX SERIES

PART 4

What Can NGINX Do?

NGINX is more than just a web server.

01 Reverse Proxy

Client → N → Backend

Hides backend services and forwards requests securely.

02 Load Balancer

Client → N → App 1, App 2, App 3

Distributes traffic across multiple servers.

03 Static File Server

N → HTML, CSS, JS, IMG

Serves static content super fast with minimal resources.

04 TLS Termination

HTTPS → N → Backend

Handles SSL/TLS and decrypts traffic before sending to app.

05 Caching

Client → N → Cache

Caches responses to reduce backend load & improve speed.

06 Rate Limiting

Protects your services from abuse & heavy traffic.

Real-World Use Case

Users → Internet → NGINX →

- /api → Backend API
- /static → Static Files
- /admin → Admin Panel
- /metrics Monitoring

One Nginx, Many Destinations!

Minimal Working Config

```
server {
  listen 80;                    # Listen on port 80
  server_name example.com;  # Domain

  location / {
    root  /var/www/html;      # Serve files
    index  index.html;        # Default file
  }

  location /api/ {
```

Simple but Powerful!

Why Engineers Love NGINX

- High performance
- Low memory usage
- Event-driven architecture
- Flexible & extensible
- Battle-tested in production

Pro Tip

@linux_dev_ops

NGINX SERIES

NGINX Explained

From Zero to Production PART 1

The web server behind millions of modern applications.

WHAT YOU'LL LEARN

- ✅ What Nginx is
- ✅ Why it was created
- ✅ How requests flow through it
- ✅ Common production use cases
- ✅ Why engineers use it

FUN FACT

NGINX (engine X) was created to solve the C10K problem — handling 10,000 concurrent connections efficiently.

NGINX is more than a web server — it's a powerful reverse proxy, load balancer, and performance booster!

Swipe to understand Nginx →